

Implementation Guide

Architecture notes, project conventions, implementation seams, and decision records.

Working draft

Architecture notes, project conventions, implementation seams, and decision records.

Purpose and scope

This index is the durable architecture and decision reference for OWA and Nexus.

Use the docs this way:

- docs/specifications.md = what exists in code today
- docs/implementation-guide.md = architecture, implementation rules, decisions, and canon-candidate direction
- docs/roadmap.md = planned work and open questions
- docs/concepts.md = shared term reference

This guide can record current implementation rules and future-facing canon candidates, but it should label those clearly and should not present them as current code truth when they are not yet implemented.

Current summary

The active architecture is:

- core/* = portable engine, schemas, packets, projections, and contracts
- runtime/* = storage, runtime orchestration, and API-facing glue
- app/* = application-layer UI, hooks, content, and shared state
- src/app/* = Expo Router route shell and API entrypoints

The current product stack is still best understood as:

- OWA App = public-facing civic surface
- Nexus Browser = graph and inspection surface
- Nexus Core = portable packet engine

Chapters

- Architecture And Layers
- Core Entities And Packet Model
- Trust, Moderation, And Policy
- Governance, Initiatives, And Decisions
- Decision Log 2026-04
- Decision Log 2026-05

Public content link contract cleanup (Pass 10A)

- Public-site actions now use shared content types from `app/public/content-types.ts` rather than component-owned action types.
- `PublicLinkTarget` supports route, external, and static download targets while preserving legacy route href fields for existing content.
- External public links are centralized in `app/public/public-links.ts` so Support, footer, and future document/download passes have one place to wire real URLs.

Docs directory and readable document shell (Pass 10B)

- The public `/docs` route now uses a directory-first shape instead of rendering the OWA Charter as a grid of principle cards.
- Readable documents are consumed from `app/public/generated/public-docs.generated.ts`, which is intentionally shaped as future build output from public Markdown source files.
- The current generated-style module is maintained by hand until the public docs build pipeline compiles Markdown, downloadable artifacts, and reader data.
- The old docs principle-grid components were removed from active use so the docs page can evolve around document directory entries, readable web documents, and static download links.

Public docs Markdown build pipeline (Pass 10C)

- Public docs source is now declared in `docs/public/public-docs.manifest.json` and compiled by `scripts/build-public-docs.mjs`.
- The build script creates readable document data in `app/public/generated/public-docs.generated.ts`, static Markdown downloads in `public/downloads/`, and hash-based internal version records in `docs/public/version-records/`.
- Source documents may be defined by `sourceDir` or ordered `sourceFiles`, allowing chapter-split docs to compile into one current public artifact per document.
- `npm run docs:build` is wired into `npm run export:web` so Railway/web exports rebuild generated docs before the Expo web build.

- Version records are for internal record-keeping only; the public docs page exposes only the current readable/downloadable document versions.

Docs reader scroll and action cleanup (Pass 10E)

- The Docs page scroll controls now target the shared PublicPageShell scroll view directly instead of using browser scrollIntoView, avoiding outer-page scroll jumps that made the footer appear to float over content.
- Document directory actions can render in a stacked layout so Download .md and disabled PDF placeholders do not overflow narrow cards.
- The readable document view returned to one continuous card with an internal outline and section dividers, rather than giving each document section its own card.
- The public footer now uses an opaque bg-public-shell background so it does not visually blend with scrolled content behind it.

Architecture And Layers

Product layers

The long-term architecture remains a three-layer stack with strict boundaries:

OWA App

The public-facing civic surface.

- OWA-native framing and navigation
- geography-first assembly experience
- place -> assembly -> deliberation -> proposals or votes -> actions -> record -> learning

Nexus Browser

The packet and graph surface.

- inspect packets
- follow links
- inspect provenance
- search and filter
- browse incoming and outgoing references

Nexus Core

The portable engine with no UI assumptions.

- packet typing and validation
- graph building
- read and write operations
- import/export bundles
- merge strategy
- trust hooks
- policy evaluation

The load-bearing rule stays the same: data is the system; platforms are adapters.

Repository boundaries

The active repo split is:

- core/* for portable packet logic, schemas, builders, interpreters, projections, and contracts
- runtime/* for persistence, runtime services, query services, auth/trust/discussion orchestration, and API glue
- app/* for application-layer components, hooks, constants, public content, and shared state
- src/app/* for the Expo Router route shell and API entrypoints

Core versus adapters

Core owns

- packet schemas and validation
- graph relationships and traversal
- import/export/merge rules
- signing and verification law
- policy evaluation hooks and deterministic business logic

Adapters own

- routing and navigation
- rendering and layout
- input UX
- device or browser integration
- orchestration around core-owned operations

Adapters should not invent parallel business logic for trust, validation, compatibility, or merge behavior.

Navigation and shell model

The intended navigation model still uses three mental axes:

- place or scope
- topic or work
- time or evidence

The current implemented Nexus slice is a practical early version of that model, with:

- dashboard
- discussions
- votes
- roles
- trust
- library
- packet explorer as a shell-level inspection workspace
- account and identity custody as wrapper-level surfaces

Core Entities And Packet Model

Packet graph foundation

The packet graph remains the most robust unifying model.

Core packet rules:

- `packet_id` is the stable logical packet identity
- `revision_id` is the immutable exact revision identity
- `parentrevisionrefs` represent DAG ancestry
- `schema_version` tracks family-shape compatibility
- edges are the shared typed relationship collection
- `authorityscoperef` and `applicablescopecrefs` stay separate
- `revision_mode` expresses family write semantics

Raw stored packets remain the historical signed fact. Adapted packets are runtime read views, and interpreted or read-model outputs are additional projections on top of those reads.

Core entities

Element

The universal identity anchor.

- person
- assembly
- organization
- service
- overlay or coordination group
- initiative

Scope

The context or lens around an element. Every person can be treated as their own scope lens through You.

Assembly

A policy-governed civic or geographic element subtype, not a separate storage primitive.

Initiative

A generic Nexus concept for policy and template lineage. OWA should be modeled as an initiative inside Nexus rather than as a hardcoded exception.

Claim

The current canonical packet family for scoped social assertions, including:

- role_association
- assembly_association
- home_locality

Current code truth:

- Claim remains a distinct packet family today
- the docs should continue to describe that as current architecture

Attestation

The current packet family for support, dispute, and other evidence-oriented signals.

Current code truth:

- claim support and dispute currently stay on Attestation
- Attestation and Claim are still distinct in code

Role

A reusable role definition packet family. Exact-scope role assertions are currently represented through Claim(kind: "role_association").

Policy

The configuration and legitimacy family for defaults, thresholds, and later execution rules.

Decision

A governance artifact family that exists in infrastructure and read surfaces, but whose real workflow semantics are still developing.

Relationships and graph semantics

The graph should continue to express relationships through typed refs and edges rather than UI-only linkage. Important relationship categories remain:

- references or depends_on
- proposes or supersedes
- belongs_to
- scoped_to
- endorsedby or vouchedby
- implements or enacts
- reports_on
- forkof or derivedfrom

Trust, Moderation, And Policy

Trust direction

Trust should remain layered, inspectable, and threshold-based rather than collapsing into a hidden score.

Current code truth:

- trust stages are currently explicit and threshold-based
- support and dispute evidence are packet-backed
- role posture is projected from scoped claims plus claim-targeted attestations

Direction:

- trust should stay scope-local
- trust should gate posting, voting, review, moderation, or role authority only through explicit policy and thresholds
- evidence and posture should remain inspectable even when defaults hide or deprioritize low-trust activity

Moderation and automoderation

Status: canon candidate

Current code truth

- moderation workflows are not yet fully implemented
- some read surfaces already expose packet-backed evidence and action visibility
- disabled placeholders are visible in several surfaces

Direction

- automoderation should be a generic feature, not a route-local trick
- default moderation baselines should come from the relevant element's policy
- user-level moderation preferences should be available as adjustable policy or settings choices, similar in spirit to write protection preferences
- moderation should primarily act as visibility projection over packet and attestation graphs before it becomes hard rejection logic

Unresolved

- which moderation controls belong in generic policy versus OWA default policy
- which interactions should remain soft projection versus hard runtime enforcement
- how much moderation preference state should be actor-level policy versus client-level shell preference

Attestations, reactions, and claims

Status: canon candidate

Current code truth

- Claim is still its own packet family
- Attestation is still its own packet family
- current discussion voting uses packet-signal attestations rather than a separate reaction family

Direction

- keep the current Claim split in code for now
- record a live design question about whether the long-term model should converge toward a broader attestation umbrella with subtypes such as:
 - reaction
 - voucher
 - objection
 - claim
- keep the semantics distinction clear even if the long-term storage model converges

Unresolved

- whether reactions deserve their own base family or should remain an attestation-like subtype
- whether vouch and flag should become explicit attestation subtypes
- whether claims should ever collapse into a generalized attestation family

This is an active modeling question, not a current implementation decision.

Governance, Initiatives, And Decisions

Initiatives

Status: canon candidate

Current code truth

- initiative-specific filtering, subscriptions, and version visibility are not yet active product behavior
- current code does not yet implement initiative-version subscriptions or official versus unofficial filtering

Direction

- Initiative should be treated as a generic Nexus concept for policy and template lineage
- OWA should be modeled as an initiative element inside Nexus, not as a hardcoded exception
- "official OWA" should mean conforming to recognized OWA dependencies, templates, and policies
- assemblies remain valid Nexus objects even when they fork or diverge from canonical OWA lineage

Unresolved

- the exact packet/API shape for initiative conformance and recognition
- how official, unofficial, derived, and forked initiative states should be encoded

Initiative versioning and subscriptions

Status: canon candidate

Direction

Initiative lineage should support version-aware adoption and viewing, including:

- current official version
- older official versions
- upgrade availability
- optional inclusion of unofficial forks

Subscriptions and default visibility should eventually support choices like:

- current official only
- all official versions
- official plus unofficial
- multiple initiatives together

Unresolved

- the exact version packet or metadata shape
- whether subscriptions belong primarily to actor policy, shell preference, or both
- how version compatibility and upgrade prompts should surface in Explorer and feeds

Assemblies, custody, and legitimacy

Status: canon candidate

Direction

- assemblies are elements with policy-governed civic semantics, not owned subsidiaries
- custody, legitimacy, policy authority, and governance outcome should remain distinct concepts
- association should remain a simple claim first; typed categories are deferred
- multi-key assembly authority, custody transition, and protected-state mutation should remain explicit later design work

Unresolved

- the exact custody and multi-key packet model
- how authority grants or keyset policies should be represented
- how passed decisions eventually authorize protected state changes

Proposals and decisions

Status: canon candidate

Current code truth

- votes remain read-only
- proposals, votes, and decisions are not yet a fully interactive trusted workflow

Direction

- decisions should begin as legitimacy records and civic signals
- policy may later allow some decisions to activate real effects
- proposals should declare their intended outcome as explicitly and programmatically as possible

Proposal outcome categories should eventually include:

- policy adoption
- mission or coordination plan
- resolution
- signal or poll
- other explicit action packages

Automatic execution should only be described as future behavior for explicitly supported built-in action handlers or canonical approval artifacts.

Unresolved

- which decision types can become effect-bearing first
- how multi-key assembly authority interacts with execution
- whether execution produces direct mutations, canonical approval packets, or both

Decision Log 2026-04

This monthly log condenses the April 2026 decisions that remain most important for current implementation and future migration work.

2026-04-08 foundations

- Packet identity uses stable packetid plus immutable revisionid.
- Revision history is a DAG with parentrevisionrefs, not a single chain.
- Packet relationships normalize into one typed edge collection.
- Element remains the identity-root family for people, assemblies, organizations, and services.
- The dedicated Nexus shell lives under /nexus/*.
- Function-first and scope-first remain one system rather than two route trees.

2026-04-09 shell and discussion shape

- Shared browser and Nexus query services over the packet graph became the projection seam.
- Active shell and scope routes moved onto packet-backed payloads instead of mock arrays.
- Discussions became a connected forum shell with feed, thread, and post workspaces.
- Feed cards became the primary thread-launch surface.
- Cursor paging and reply-branch controls became first-class route behavior.

2026-04-10 identity and auth

- Every graph-writing actor became a cryptographic Element(kind: "person"), including guests.
- Claimed auth uses local keys plus server challenge sessions rather than server-owned identity truth.
- Claimed auth now supports passkeys, rotating sessions, and explicit write-approval modes.
- Identity ceremonies moved into the Nexus shell.
- Attestation replaced packet votes as the reusable trust edge.

2026-04-17 architecture and trust pivot

- The final source roots became core, runtime, and app, with src/app as the Expo Router shell.
- Trust replaced Account as the top-level function surface.
- You became a first-class personal scope lens.
- Roles became a dedicated scope workspace.
- Role, Claim, and packet compatibility infrastructure became first-class runtime concerns.

2026-04-18 to 2026-04-20 locality and identity hardening

- Legacy signed identities remained valid after the roles-and-claims refactor.
- Home locality became the mounted geographic truth, while follows became shell preferences.
- Locality search and locality creation hardened around canonical geographic assembly flows.
- Location disclosure remained separate from mounted home-locality truth.

2026-04-23 and 2026-04-24 compatibility and fortress corridor

- Schema compatibility moved into a real raw/adapted/read-for-write pipeline.
- Fortress pass 1 moved discussion write admission into core mutation law.
- Fortress pass 2 introduced the shared prepare/sign/finalize corridor and packet-backed write-lock policy updates.

2026-04-25 to 2026-04-29 packet floor and verification

- Packet compatibility coverage is now classified explicitly.
- Legacy write seams were sealed before discussion-family cleanup.
- Raw packet signatures verify before adaptation.
- Live and near-live packet families now share one generic builder floor.

2026-04-30 surface-level runtime contracts

- Discussion workspaces now project action affordances through the shared NexusActionState and NexusActionIntentDescriptor contract.
- Packet Explorer became a shell-level read-only inspection workspace.

Decision Log 2026-05

This monthly log condenses the May 2026 decisions that remain most important for current public-surface and documentation shape.

2026-05-01 public surface convergence

- Shared public actions and section shells now route through one surface contract.
- PublicCardFrame separates shared graphic treatment from page-owned content layout.
- Support and Docs now share the same public frame and panel system.
- Public card frames gained ambient generated backgrounds owned by the shared frame layer.
- About surfaces joined the shared public frame path without absorbing card-specific animation rules.
- Public sections now use the universal card animation path.

- Public theme cleanup centralized lingering route-local styling values into named public seams.

2026-05 chaptering follow-up

- Canon docs are being split into index files plus shallow chapter folders to reduce edit cost, preserve top-level link stability, and keep current truth, durable architecture, and roadmap work separated cleanly.

2026-05 shell honesty foundation

- Nexus now uses a shell-level early-access gate as a session-scoped entry warning rather than leaving public-facing instability as unstated product lore.
- The gate is intentionally lightweight and UI-owned: it is not packet-backed, not actor-specific, and not yet a tutorial flow.
- The same shell seam should later be able to host onboarding, release notes, or other blocking notices without redesigning the overlay stack.

2026-05 feature-status explainers

- Disabled and partial Nexus controls now route through a centralized feature-status registry instead of scattering one-off placeholder copy through route files.
- The first-pass status taxonomy is intentionally small: comingsoon, readonly, partial, and custom, with registry-owned overrides when a control needs more specific wording.
- Nexus shell now hosts one explainer card overlay at a time so disabled controls can stay compact while still opening truthful contextual explanations above Explorer, Library, Votes, and future surfaces.
- The shared NexusActionButton seam remains semantically disabled for these controls, but can now open an explainer instead of acting like a dead button when a feature-status id is attached.

2026-05 Explorer and shell mobile cleanup

- NexusInlineSelect now renders its open menu on a true overlay layer instead of trusting local stacking order, so Explorer View as menus can sit above adjacent header controls.
- Packet Explorer tab decks now behave like normal wrapped rows until they exceed three rows, and only then become scrollable.
- Narrow-screen shell entry now prioritizes immediate access to the real menu rails by opening both rails when the tray is opened, instead of preserving collapsed desktop rail state inside the mobile tray.
- That same mobile tray now sizes itself to the currently visible rails and closes entirely when both rails are minimized.

- Library packet highlighting is now treated as Explorer-linked state rather than a permanently sticky URL-only cue, so stale selection clears when the corresponding Explorer packet tab disappears.
- Explorer resize now lives in a dedicated Explorer-specific desktop drag controller rather than the main overlay coordinator, and uses global window mouse tracking plus temporary drag capture instead of a moving PanResponder handle.
- Shared browser packet projections now route through the core packet-title projection path instead of falling back directly to raw packet ids, which keeps Explorer and other packet-inspection surfaces from flipping loaded titles into percent-encoded route strings.
- Explorer reading now uses one primary content scroll surface with local collapsible header bands, while Close tabs has moved out of the shell command row and into the packet-tab deck as a packet-management utility.
- Explorer collapse controls were then tightened so the bands do not reserve their own chevron rows: collapsed sections disappear fully, and their restore actions move into the top shell command row as Packets and Views.

2026-05 Packet Explorer export workbench

- Packet Explorer Home is now explicitly split into Search and Export sub-tabs so future Explorer capabilities can grow from one stable workspace without collapsing back into the main packet inspector.
- The first live portability seam is export only: Search stays present but non-live, while Export becomes the active raw-json workbench.
- Packet export defaults to Raw packet, which means the current preferred revision envelope only; bundle wrapping is an explicit opt-in instead of the default artifact shape.
- Bundle export remains additive and transport-oriented: the envelope now records export metadata such as mode, root refs, counts, title, and note without changing packet schemas or mutating the packet envelopes inside packets.
- Phase 1 bundle scopes stay concrete and bounded to current graph/scoping contracts: packet history, outgoing references, incoming referrers, scope-stack ancestry, their combined union, and full local-store export as a separate node-level snapshot tool.
- Preview and download now share one export builder seam with explicit size guardrails so Explorer can show inline JSON only for small payloads while still allowing larger .json bundle downloads without inventing a second export format.

2026-05 Packet Explorer import workbench

- Packet Explorer Home now expands to Search, Import, and Export, with Search-card import shortcuts routing into the new Import workspace rather than spawning separate packet-vs-bundle pages.

- Import is intentionally a two-step flow: Analyze first, then Commit, so pasted or uploaded JSON does not mutate the local store until the runtime preview says the payload is structurally safe.
- Phase 2 import stays generic and backward-tolerant by accepting raw packet envelopes, exported bundle envelopes with packets, legacy bundle envelopes with revisions, and raw arrays, all normalized onto the existing bundle-import substrate.
- Web import now includes a browser .json file picker, but paste remains the universal fallback so the workflow still works outside the browser without adding a cross-platform file abstraction in this pass.
- Import analysis is structural rather than trust-verifying: it reports invalid entries, duplicates, missing parent revisions, family conflicts, affected packets, and likely open targets, then blocks commit on unsafe inputs instead of offering partial-import toggles.
- Post-import repair now preserves local preferred-head intent when divergence appears: if a newly imported branch creates multiple heads and the old preferred head still exists, Explorer restores that preferred revision instead of silently replacing it with the last imported head.

2026-05 Packet Explorer live search and export lookup

- Packet Explorer Search is now a real manual discovery workspace instead of a placeholder card, and its state stays Home-owned so query text, grouped results, and the active category survive normal Explorer tab movement while the session stays open.
- Search intentionally stays preferred/current only in this pass: it reads from the shared packet search index rather than adding FTS, graph traversal search, or historical revision fanout into the main result set.
- Exact revision-id search is still supported through a separate revision resolver seam on the packet store, which lets Explorer map historical revision IDs back to the owning packet while keeping the displayed packet card anchored to the current preferred revision surface.
- Search results are grouped into Direct, Name, and Text, with deterministic ranking and lightweight Open / Export routing instead of turning the first live pass into a broader action hub.
- Export now includes a compact active lookup when no packet target is preloaded, but that lookup intentionally remains narrower than the full Search workspace so Explorer keeps one discovery-oriented search surface and one operational packet-picker.

2026-05 Packet Explorer workspace cleanup

- Home workspace navigation has now moved out of the Home body and into the shared inspector row, so Search, Import, and Export behave like first-class Explorer modes rather than nested content tabs.

- The packet primary rail and the Home workspace rail now share one compact attached-tab treatment, dropping the previous secondary tab copy so the inspector band can stay horizontally accessible without running off-screen.
- Home-only dead controls were removed instead of being left as placeholder chrome, and the Search workspace now keeps only its core manual finder actions rather than duplicating Import navigation through extra shortcut buttons.
- Packet export was re-centered around direct packet lookup when no target is preloaded, with the packet-picker promoted ahead of explanatory copy so the export workspace feels operational instead of instructional.
- Search now defensively tolerates a missing runtime revision resolver by falling back to the preferred/current packet index path instead of failing the whole search request.

2026-05 Packet Explorer final polish

- Link traversal in Explorer is now explicit instead of ambiguous: grouped links expose Open in new tab, Open in current tab, and View in Library, while current-tab retargeting preserves the existing inspector state and refreshes the active packet identity fields.
- Packet tab decks now present Home as the workspace tab label, use middle truncation for packet titles, and surface desktop hover metadata so long packet titles and revision context stay discoverable without widening the deck.
- Home Export now includes an explicit Cancel reset path for packet-scoped export so preloaded packet targets can be cleared without leaving the Export workspace or disturbing the separate full local store export surface.
- Search preview caps and category browsing are now separated: All remains an 8-result grouped preview surface, while focused Direct, Name, and Text views page through larger result sets with server-backed 25-item pages.